Improving Area and Resource Utilization Lab

Objectives

After completing this lab, you will be able to:

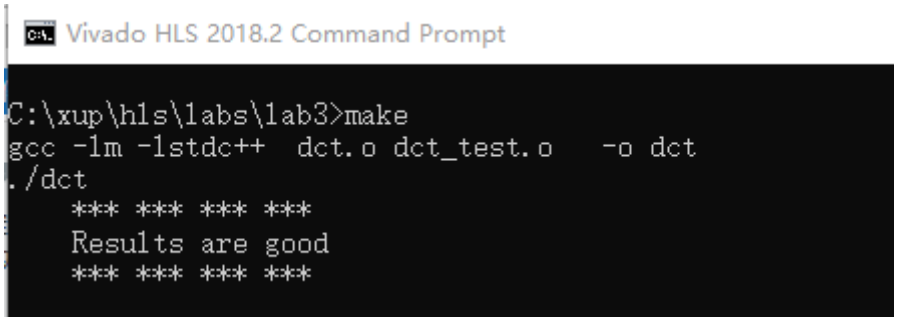
- Add directives in your design
- Improve performance using PIPELINE directive
- Distinguish between DATAFLOW directive and Configuration Command functionality
- Apply memory partitions techniques to improve resource utilization

Steps

Validate the Design from Command Line

Validate your design from Vivado HLS command line.

1. Click **Start > Xilinx Design Tools > Vivado HLS 2018.2 Command Prompt**.
2. In the *Vivado HLS Command Prompt* window, change directory to **c:\xup\hls\labs\lab3**.
   A self-checking program (dct_test.c) is provided. Using that we can validate the design. A Makefile is also provided. Using the Makefile, the necessary source files can be compiled and the compiled program can be executed.
3. In the *Vivado HLS Command Prompt* window, type **make** to compile and execute the program.



Validating the design

   Note that the source files (dct.c and dct_test.c are compiled, then dct executable program was created, and then it was executed. The program tests the design and outputs Results are good message.

4. Close the command prompt window by typing **exit**.

Create a New Project

Create a new project in Vivado HLS GUI targeting xc7z020clg400-1.

1. Select **Start > Xilinx Design Tools > Vivado HLS 2018.2**.
2. In the Vivado HLS GUI, select **File > New Project**. The New Vivado HLS Project wizard opens.
3. Click **Browse...** button of the Location field and browse to **c:\xup\hls\labs\lab3** and then click **OK**.
4. For Project Name, type **dct.prj** and click **Next**.
5. In the Add/Remove Files for the source files, type **dct** as the function name (the provided source file contains the function, to be synthesized, called dct).
6. Click the **Add Files...** button, select **dct.c** file from the **c:\xup\hls\labs\lab3** folder, and then click **Open**.
7. Click **Next**.
8. In the *Add/Remove Files* for the testbench, click the **Add Files...** button, select **dct_test.c**, **in.dat**, **out.golden.dat** files from the **c:\xup\hls\labs\lab3** folder and click **Open**.
9. Click **Next**.
10. In the Solution Configuration page, leave Solution Name field as solution1 and set the clock period as 10. Leave Uncertainty field blank.
11. Click on Part's Browse button, and select the following filters, using the Parts Specify option, to select **xc7z020clg400-1**.
12. Click **Finish**.
13. Double-click on the **dct.c** under the source folder to open its content in the information pane.

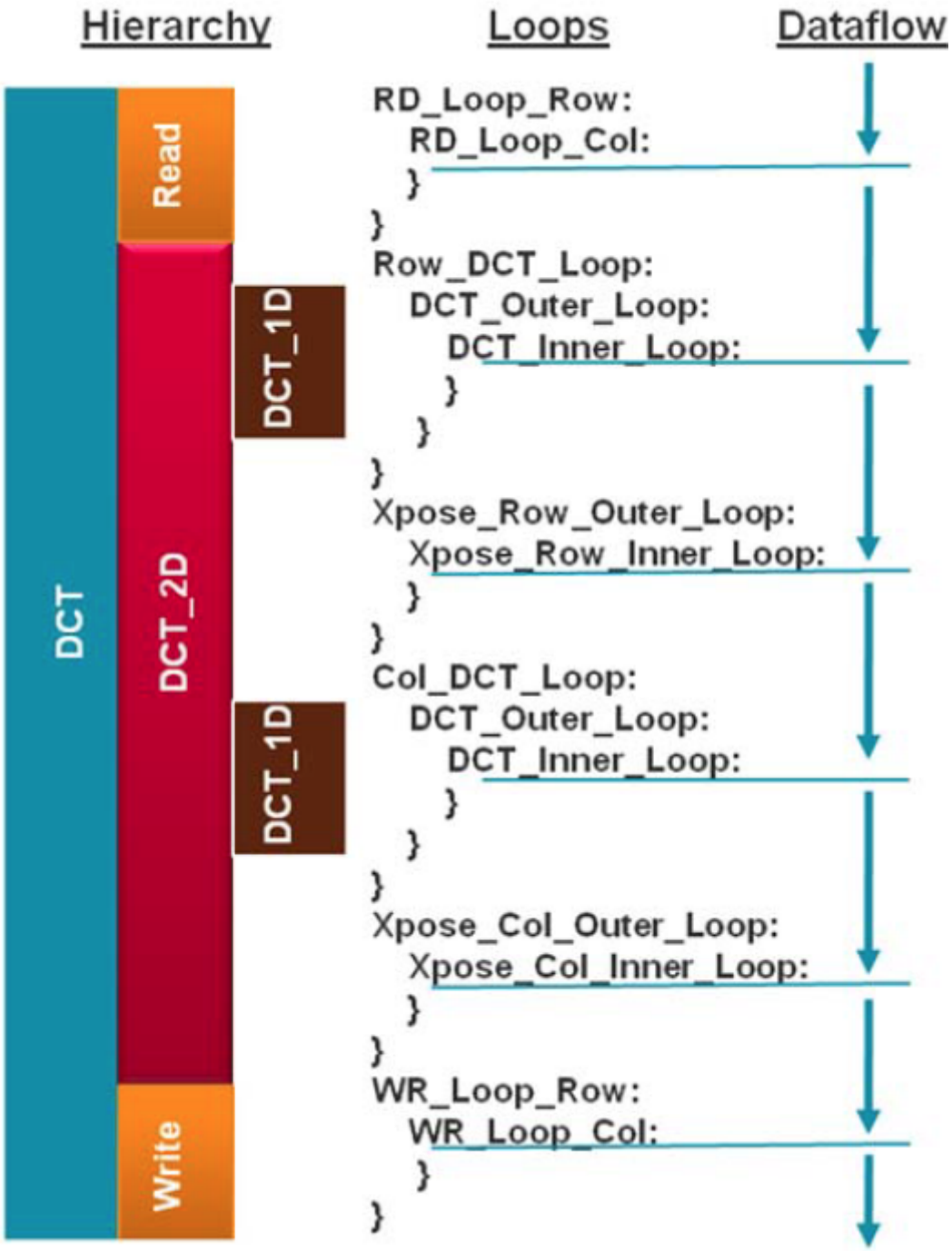
```
78 void dct(short input[N], short output[N])
79 {
80
81    short buf_2d_in[DCT_SIZE][DCT_SIZE];
82    short buf_2d_out[DCT_SIZE][DCT_SIZE];
83
84    // Read input data. Fill the internal buffer.
85    read_data(input, buf_2d_in);
86
87    dct_2d(buf_2d_in, buf_2d_out);
88
89    // Write out the results.
90    write_data(buf_2d_out, output);
91 }
```

The design under consideration

The top-level function *dct*, is defined at line 78. It implements 2D DCT algorithm by first processing each row of the input array via a 1D DCT then processing the columns of the resulting array through the same 1D DCT. It calls read_data, dct_2d, and write_data functions.

The read_data function is defined at line 54 and consists of two loops – RD_Loop_Row and RD_Loop_Col. The write_data function is defined at line 66 and consists of two loops to perform writing the result. The dct_2d function, defined at line 23, calls dct_1d function and

performs transpose.
Finally, dct_1d function, defined at line 4, uses dct_coeff_table and performs the required function by implementing a basic iterative form of the 1D Type-II DCT algorithm. Following figure shows the function hierarchy on the left-hand side, the loops in the order they are executes and the flow of data on the right-hand side.



Design hierarchy and dataflow

Synthesize the Design

Synthesize the design with the defaults. View the synthesis results and answer the question listed in the detailed section of this step.

1. Select **Solution > Run C Synthesis > Active Solution** or click on the button on tools bar to start the synthesis process.
2. When synthesis is completed, several report files will become accessible and the Synthesis Results will be displayed in the information pane.
   Note that the Synthesis Report section in the Explorer view only shows dct_1d.rpt, dct_2d.rpt, and dct.rpt entries. The read_data and write_data functions reports are not listed. This is because these two functions are inlined. Verify this by scrolling up into the Vivado HLS Console view.

```
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.c:85) automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct' (dct.c:90) automatically.
INFO: [HLS 200-111] Finished Checking Synthesizability Time (s): cpu = 00:00:02 ; elapsed
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.c:85) automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct' (dct.c:90) automatically.
```

Inlining of read_data and write_data functions

3. The *Synthesis Report* shows the performance and resource estimates as well as estimated latency in the design. Note that the design is not optimized nor is pipelined.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	6.508	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
3959	3959	3959	3959	none

Detail

Instance

Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- RD_Loop_Row	144	144	18	-	-	8	no
+ RD_Loop_Col	16	16	2	-	-	8	no
- WR_Loop_Row	144	144	18	-	-	8	no
+ WR_Loop_Col	16	16	2	-	-	8	no

Synthesis report

4. Using scroll bar on the right, scroll down into the report and answer the following question.

   Question 1
   Answer the following question:
   Estimated clock period:
   Worst case latency:

Number of DSP48E used:
Number of BRAMs used:
Number of FFs used:
Number of LUTs used:

5. The report also shows the top-level interface signals generated by the tools.

Interface

Summary

RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	dct	return value
ap_rst	in	1	ap_ctrl_hs	dct	return value
ap_start	in	1	ap_ctrl_hs	dct	return value
ap_done	out	1	ap_ctrl_hs	dct	return value
ap_idle	out	1	ap_ctrl_hs	dct	return value
ap_ready	out	1	ap_ctrl_hs	dct	return value
input_r_address0	out	6	ap_memory	input_r	array
input_r_ce0	out	1	ap_memory	input_r	array
input_r_q0	in	16	ap_memory	input_r	array
output_r_address0	out	6	ap_memory	output_r	array
output_r_ce0	out	1	ap_memory	output_r	array
output_r_we0	out	1	ap_memory	output_r	array
output_r_d0	out	16	ap_memory	output_r	array

Generated interface signals

You can see ap_clk, ap_rst are automatically added. The ap_start, ap_done, ap_idle, and ap_ready are top-level signals used as handshaking signals to indicate when the design is able to accept next computation command (ap_idle), when the next computation is started (ap_start), and when the computation is completed (ap_done). The top-level function has input and output arrays, hence an ap_memory interface is generated for each of them.

6. Open *dct_1d.rpt* and *dct_2d.rpt* files either using the *Explorer* view or by using a hyperlink at the bottom of the *dct.rpt* in the information view. The report for dct_2d clearly indicates that most of this design cycles (3668) are spent doing the row and column DCTs. Also the dct_1d report indicates that the latency is 209 clock cycles ((24+2)*8+1).

Run Co-Simulation

Run the Co-simulation, selecting Verilog. Verify that the simulation passes.

1. Select **Solution > Run C/RTL Co-simulation** or click on the button on tools bar to open the dialog box so the desired simulations can be run.
A C/RTL Co-simulation Dialog box will open.

2. Select the **Verilog** option, and click **OK** to run the Verilog simulation using XSIM simulator. The RTL Co-simulation will run, generating and compiling several files, and then simulating the design. In the console window you can see the progress and also a message that the test is passed.

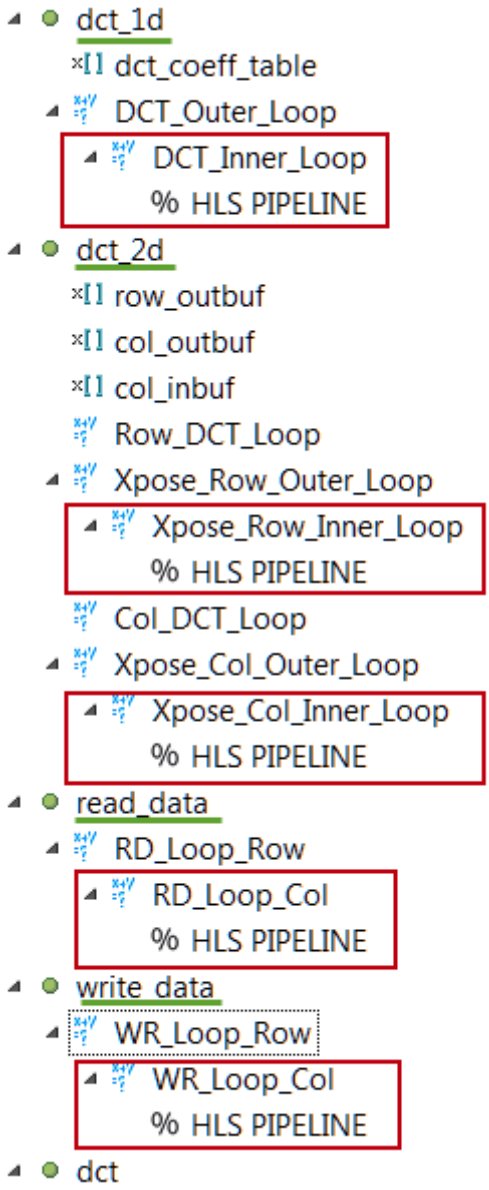
```
INFO: [COSIM 212-316] Starting C post checking ...
    *** *** *** ***
    Results are good
    *** *** *** ***
INFO: [COSIM 212-1000] *** C/RTL co-simulation finished: PASS ***
Finished C/RTL cosimulation.
```

RTL Co-Simulation results

Apply PIPELINE Directive

Create a new solution by copying the previous solution settings. Apply the PIPELINE directive to DCT_Inner_Loop, Xpose_Row_Inner_Loop, Xpose_Col_Inner_Loop, RD_Loop_Col, and WR_Loop_Col. Generate the solution and analyze the output.

1. Select **Project > New Solution** or click on the  button from the tools bar buttons.
2. A *Solution Configuration* dialog box will appear. Click the **Finish** button (with copy from Solution1 selected).
3. Make sure that the **dct.c** source is opened in the information pane and click on the **Directive** tab.
4. Select **DCT_Inner_Loop** of the dct_1d function in the *Directive* pane, right-click on it and select **Insert Directive**...
5. A pop-up menu shows up listing various directives. Select **PIPELINE** directive.
6. Leave II (Initiation Interval) blank as Vivado HLS will try for an II=1, one new input every clock cycle.
7. Click **OK**.
8. Similarly, apply the **PIPELINE** directive to **Xpose_Row_Inner_Loop** and **Xpose_Col_Inner_Loop** of the dct_2d function, and **RD_Loop_Col** of the read_data function, and **WR_Loop_Col** of the write_data function. At this point, the Directive tab should look like as follows.



PIPELINE directive applied

9. Click on the **Synthesis** button.
10. When the synthesis is completed, select **Project > Compare Reports...** to compare the two solutions.
11. Select *Solution1* and *Solution2* from the *Available Reports*, click on the **Add>>** button, and then click **OK**.
12. Observe that the latency reduced from *3959* to *1851* clock cycles.

Performance Estimates

Timing (ns)

Clock		solution2	solution1
ap_clk	Target	10.00	10.00
	Estimated	8.23	6.51

Latency (clock cycles)

		solution2	solution1
Latency	min	1851	3959
	max	1851	3959
Interval	min	1851	3959
	max	1851	3959

Performance comparison after pipelining

13. Scroll down in the comparison report to view the resources utilization. Observe that the FFs and/or LUTs utilization increased whereas BRAM and DSP48E remained same.

Utilization Estimates

	solution2	solution1
BRAM_18K	5	5
DSP48E	1	1
FF	256	278
LUT	1286	982

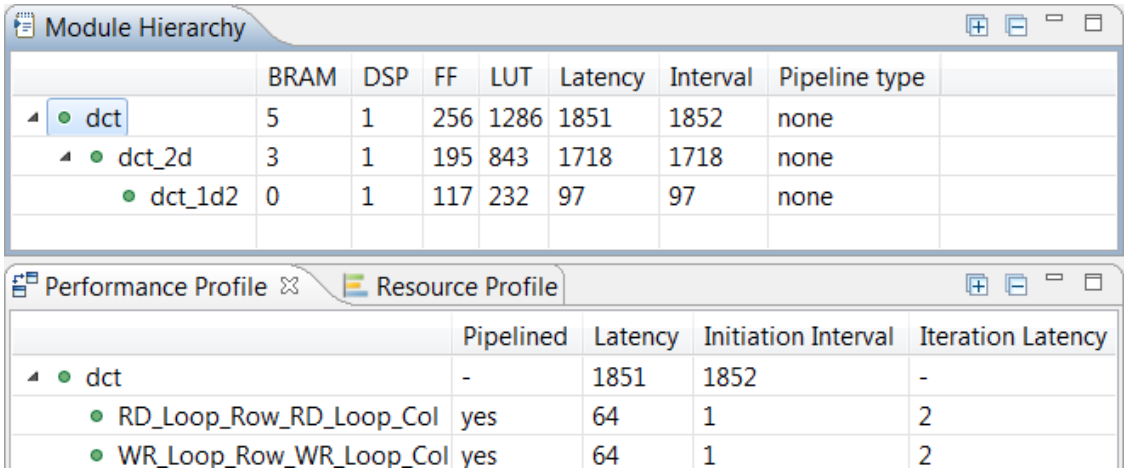
Resources utilization after pipelining

Open the Analysis perspective and determine where most of the clock cycles are spend, i.e. where the large latencies are.

1. Click on the *Open Analysis Perspective* button.
2. In the *Module Hierarchy* pane, select the **dct** entry and observe the **RD_Loop_Row_RD_Loop_Col** and **WR_Loop_Row_WR_Loop_Col** entries in the *Performance Profile* pane. These are two nested loops flattened and given the new names formed by appending inner loop name to the outer loop name. You can also verify this by looking in the *Console* view message.

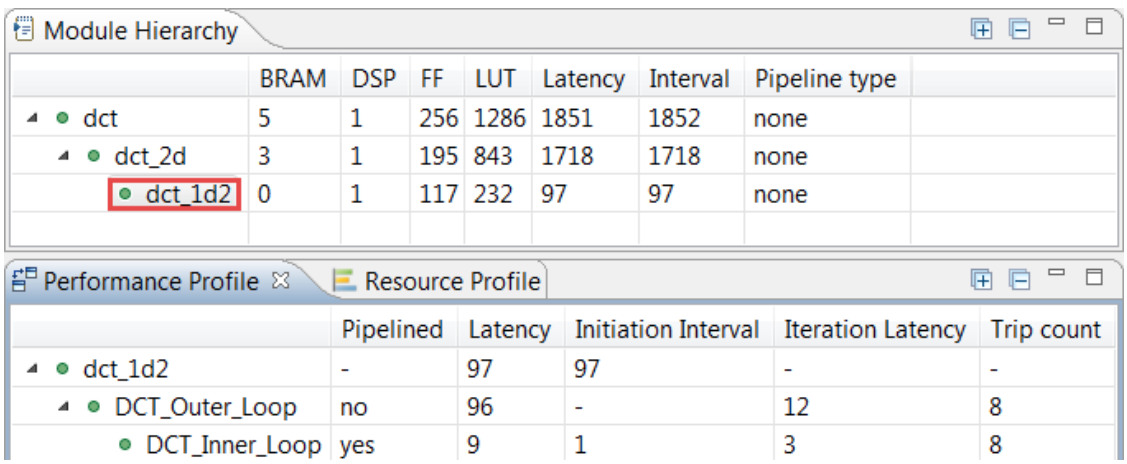
```
INFO: [HLS 200-10] Checking synthesizability ...
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.c:85) automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct' (dct.c:90) automatically.
INFO: [HLS 200-111] Finished Checking Synthesizability Time (s): cpu = 00:00:02 ; elapsed = 00:00:08 . Memory (MB):
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.c:85) automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct' (dct.c:90) automatically.
INFO: [HLS 200-111] Finished Pre-synthesis Time (s): cpu = 00:00:03 ; elapsed = 00:00:09 . Memory (MB): peak = 117.
INFO: [XFORM 203-541] Flattening a loop nest 'Xpose_Row_Outer_Loop' (dct.c:38:1) in function 'dct_2d'.
INFO: [XFORM 203-541] Flattening a loop nest 'Xpose_Col_Outer_Loop' (dct.c:49:1) in function 'dct_2d'.
WARNING: [XFORM 203-542] Cannot flatten a loop nest 'DCT_Outer_Loop' (dct.c:13:67) in function 'dct_1d' :
WARNING: [XFORM 203-542] the outer loop is not a perfect loop because there is nontrivial logic in the loop latch.
INFO: [XFORM 203-541] Flattening a loop nest 'RD_Loop_Row' (dct.c:59:67) in function 'dct'.
INFO: [XFORM 203-541] Flattening a loop nest 'WR_Loop_Row' (dct.c:71:67) in function 'dct'.
```

The console view content indicating loops flattening



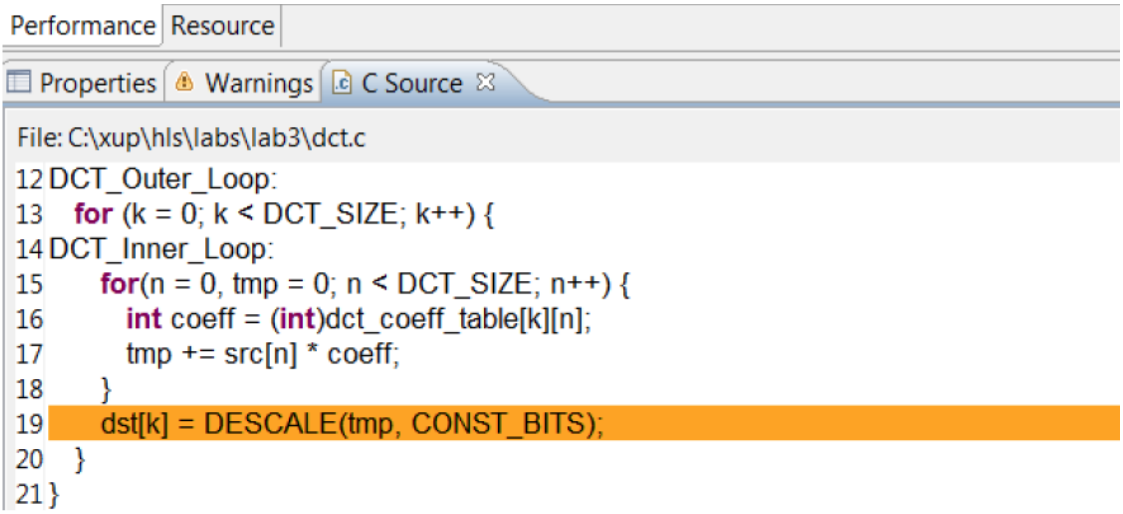
The performance profile at the dct function level

3. In the *Module Hierarchy* pane, expand **dct > dct_2d**. Notice that the most of the latency occurs is
   in **dct_2d** function.
4. In the *Module Hierarchy* pane, notice that there still hierarchy exists in the **dct_2d** module. Expand
   dct > dct_2d > dct 1d2, and select the **dct_1d2** entry.



The dct_1d function performance profile

5. In the *Performance Profile* pane, select the **DCT_Inner_Loop** entry, right-click on the **node_60** (write) block in the **Schedule Viewer**, and select **Goto Source**. Notice that line 19 is highlighted which is preventing the flattening of the DCT_Outer_Loop.

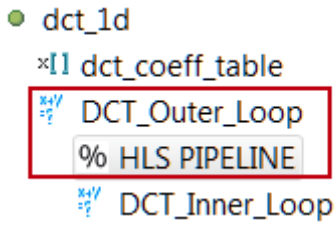


Understanding what is preventing DCT_Outer_Loop flattening

6. Switch to the *Synthesis* perspective.

Create a new solution by copying the previous solution settings. Apply fine-grain parallelism of performing multiply and add operations of the inner loop of dct_1d using PIPELINE directive by moving the PIPELINE directive from inner loop to the outer loop of dct_1d. Generate the solution and analyze the output.

1. Select **Project > New Solution**.
2. A *Solution Configuration* dialog box will appear. Click the **Finish** button (with Solution2 selected).
3. Select **PIPELINE** directive of **DCT_Inner_Loop** of the **dct_1d** function in the Directive pane, right-click on it and select **Remove** Directive.
4. Select **DCT_Outer_Loop** of the **dct_1d** function in the Directive pane, right-click on it and select **Insert Directive...**
5. A pop-up menu shows up listing various directives. Select **PIPELINE** directive.
6. Click **OK**.



PIPELINE directive applied to DCT_Outer_Loop

By pipelining an outer loop, all inner loops will be unrolled automatically (if legal), so there is no need to explicitly apply an UNROLL directive to DCT_Inner_Loop. Simply move the pipeline to the outer loop: the nested loop will still be pipelined but the operations in the inner-loop body will operate concurrently.

7. Click on the **Synthesis** button.
8. When the synthesis is completed, select **Project > Compare Reports...** to compare the two solutions.
9. Select Solution2 and Solution3 from the **Available Reports**, click on the **Add>>** button, and then click **OK**.
10. Observe that the latency reduced from *1851* to *875* clock cycles.

Performance Estimates

Timing (ns)

Clock		solution2	solution3
ap_clk	Target	10.00	10.00
	Estimated	7.883	9.400

Latency (clock cycles)

		solution2	solution3
Latency	min	1851	875
	max	1851	875
Interval	min	1851	875
	max	1851	875

Performance comparison after pipelining

11. Scroll down in the comparison report to view the resources utilization. Observe that the utilization of all resources (except BRAM) increased. Since the DCT_Inner_Loop was unrolled, the parallel computation requires 8 DSP48E.

Utilization Estimates

	solution2	solution3
BRAM_18K	5	5
DSP48E	1	8
FF	256	678
LUT	1226	1375

Resources utilization after pipelining

12. Open **dct_1d2** report and observe that the pipeline initiation interval (II) is four (4) cycles, not one (1) as might be hoped and there are now 8 BRAMs being used for the coefficient table. Looking closely at the synthesis log, notice that the coefficient table was automatically partitioned, resulting in 8 separate ROMs: this helped reduce the latency by keeping the unrolled computation loop fed, however the input arrays to the dct_1d function were not automatically partitioned.
    The reason the II is four (4) rather than the eight (8) one might expect, is because Vivado HLS automatically uses dual-port RAMs, when beneficial to scheduling operations.

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	8	-	-
Expression	-	-	0	186
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	0	-	119	16
Multiplexer	-	-	-	125
Register	-	-	420	-
Total	0	8	539	327
Available	280	220	106400	53200
Utilization (%)	0	3	~0	~0

Increased resource utilization of dct_1d

```
INFO: [XFORM 203-502] Unrolling all sub-loops inside loop 'DCT_Outer_Loop' (dct.c:13) in function 'dct_1d' for pipelining.
INFO: [XFORM 203-501] Unrolling loop 'DCT_Inner_Loop' (dct.c:15) in function 'dct_1d' completely.
INFO: [XFORM 203-102] Partitioning array 'dct_coeff_table' in dimension 2 automatically.
INFO: [XFORM 203-602] Inlining function 'read_data' into 'dct' (dct.c:85) automatically.
INFO: [XFORM 203-602] Inlining function 'write_data' into 'dct' (dct.c:90) automatically.
```

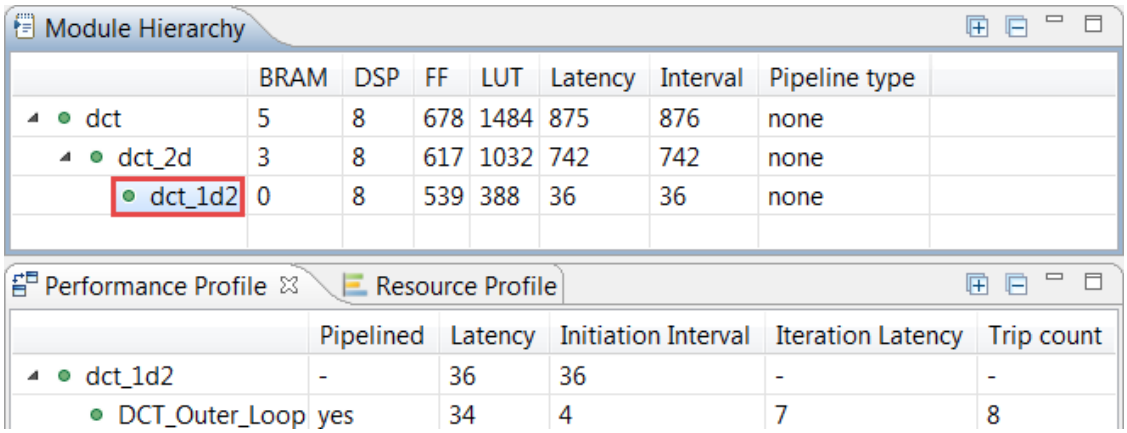
Automatic partitioning of dct_coeff_table

```
INFO: [HLS 200-10] -- Implementing module 'dct_1d2'
INFO: [HLS 200-10] ----------------------------------------------------------------
INFO: [SCHED 204-11] Starting scheduling ...
INFO: [SCHED 204-61] Pipelining loop 'DCT_Outer_Loop'.
WARNING: [SCHED 204-69] Unable to schedule 'load' operation ('src_load_5', dct.c:17) on array 'src' due to limited memory ports.
INFO: [SCHED 204-61] Pipelining result: Target II: 1, Final II: 4, Depth: 7.
WARNING: [SCHED 204-21] Estimated clock period (9.4ns) exceeds the target (target clock period: 10ns, clock uncertainty: 1.25ns,
WARNING: [SCHED 204-21] The critical path consists of the following:
        'mul' operation ('tmp_7_7', dct.c:17) (3.36 ns)
        'add' operation ('tmp7', dct.c:19) (3.02 ns)
        'add' operation ('tmp6', dct.c:19) (3.02 ns)
```

Initiation interval of 4

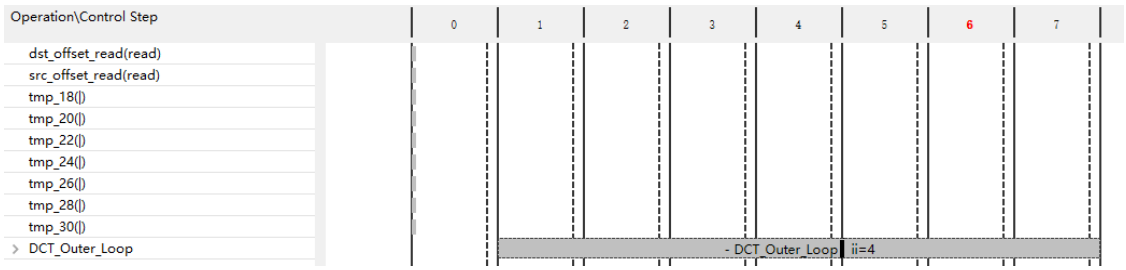
Perform design analysis by switching to the Analysis perspective and looking at the dct_1d2 performance view.

1. Switch to the Analysis perspective, expand the Module Hierarchy entries, and select the **dct_1d2** entry.
2. Expand, if necessary, the **Profile** tab entries and notice that the DCT_Outer_Loop is now pipelined and there is no DCT_Inner_Loop entry.



DCT_Outer_Loop flattening

3. Select the **dct_1d2** entry in the *Module Hierarchy* pane and observe that the DCT_Outer_Loop spans over eight states in the Performance view.



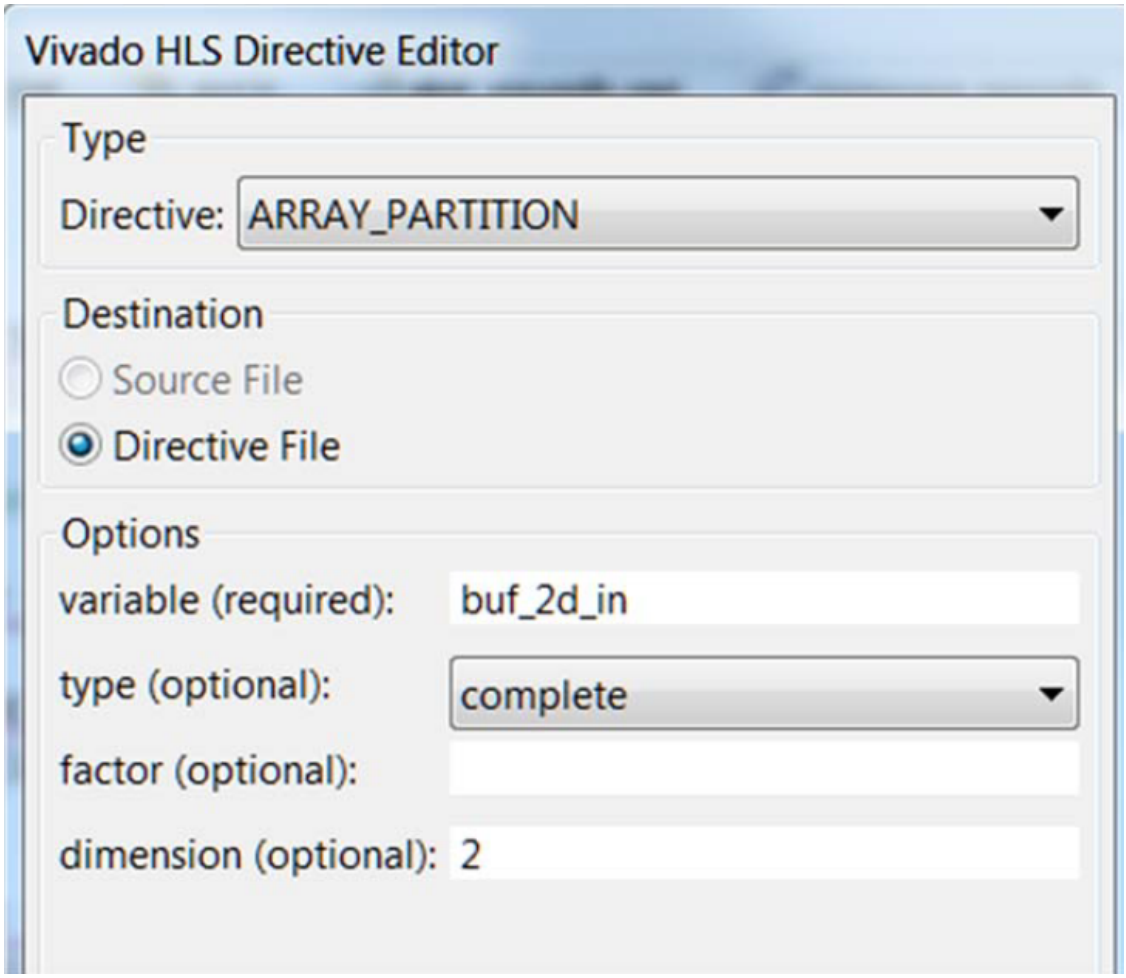
The Performance view of the DCT_Outer_Loop function

4. Switch to the *Synthesis* perspective.

Improve Memory Bandwidth

Create a new solution by copying the previous solution (Solution3) settings. Apply ARRAY_PARTITION directive to buf_2d_in of dct (since the bottleneck was on src port of the dct_1d function, which was passed via in_block of the dct_2d function, which in turn was passed via buf_2d_in of the dct function) and col_inbuf of dct_2d. Generate the solution.

1. Select **Project > New Solution** to create a new solution.
2. A *Solution Configuration* dialog box will appear. Click the **Finish** button (with Solution3 selected).
3. With dct.c open, select **buf_2d_in** array of the dct function in the Directive pane, right-click on it and select **Insert Directive...**
   The buf_2d_in array is selected since the bottleneck was on src port of the dct_1d function, which was passed via in_block of the dct_2d function, which in turn was passed via buf_2d_in of the **dct** function).
4. A pop-up menu shows up listing various directives. Select **ARRAY_PARTITION** directive.
5. Make sure that the type is complete. Enter **2** in the dimension field and click **OK**.



Applying ARRAY_PARTITION directive to memory buffer

6. Similarly, apply the **ARRAY_PARTITION** directive with dimension of 2 to the **col_inbuf** array.
7. Click on the **Synthesis** button.
8. When the synthesis is completed, select **Project > Compare Reports...** to compare the two solutions.
9. Select *Solution3* and *Solution4* from the Available Reports, and click on the **Add>>** button.
10. Observe that the latency reduced from *875* to *509* clock cycles.

Performance Estimates

Timing (ns)

Clock		solution3	solution4
ap_clk	Target	10.00	10.00
	Estimated	9.400	9.400

Latency (clock cycles)

		solution3	solution4
Latency	min	875	509
	max	875	509
Interval	min	875	509
	max	875	509

Performance comparison after array partitioning

11. Scroll down in the comparison report to view the resources utilization. Observe the increase in the FF resource utilization (almost double).

Utilization Estimates

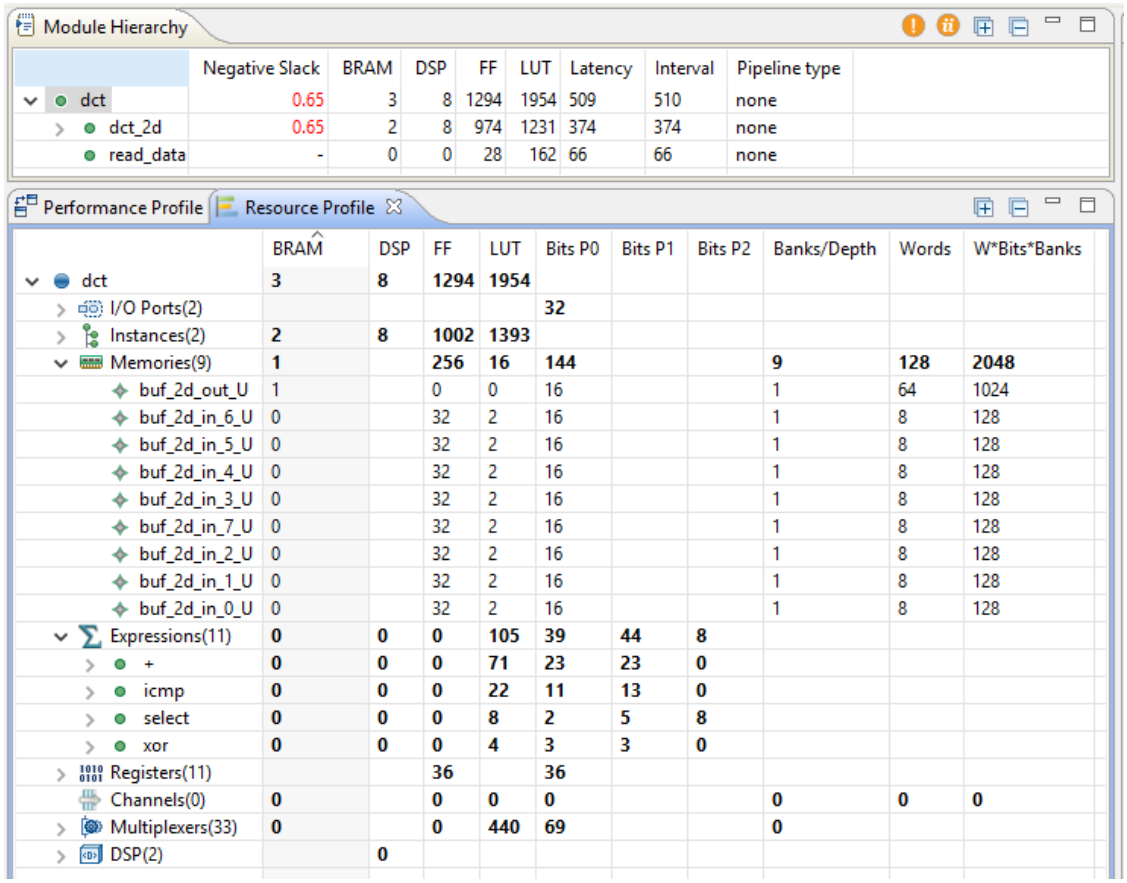
	solution3	solution4
BRAM_18K	5	3
DSP48E	8	8
FF	678	1294
LUT	1375	1954

Resources utilization after array partitioning

12. Expand the Loop entry in the **dct.rpt** entry and observe that the Pipeline II is now 1.

Perform resource analysis by switching to the Analysis perspective and looking at the dct resources profile view.

1. Switch to the *Analysis* perspective, expand the *Module Hierarchy* entries, and select the **dct** entry.
2. Select the **Resource Profile** pane.
3. Expand the **Memories** and **Expressions** entries and observe that the most of the resources are consumed by instances. The buf_2d_in array is partitioned into multiple memories and most of the operations are done in addition and comparison.



Resource profile after partitioning buffers

4. Switch to the *Synthesis* perspective.

Apply DATAFLOW Directive

Create a new solution by copying the previous solution (Solution4) settings. Apply the DATAFLOW directive to improve the throughput. Generate the solution and analyze the output.

1. Select **Project > New Solution**.
2. A *Solution Configuration* dialog box will appear. Click the **Finish** button (with Solution4 selected).
3. Close all inactive solution windows by selecting **Project > Close Inactive Solution Tabs**.
4. Select function **dct** in the directives pane, right-click on it and select **Insert Directive...**
5. Select **DATAFLOW** directive to improve the throughput.
6. Click on the *Synthesis* button.
7. When the synthesis is completed, the synthesis report is automatically opened.
8. Observe that dataflow type pipeline throughput is listed in the "Performance Estimates*

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	9.40	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
508	508	375	375	dataflow

Performance estimate after DATAFLOW directive applied

- The Dataflow pipeline throughput indicates the number of clock cycles between each set of inputs reads (interval parameter). If this value is less than the design latency it indicates the design can start processing new inputs before the currents input data are output.
- Note that the dataflow is only supported for the functions and loops at the top-level, not those
  which are down through the design hierarchy. Only loops and functions exposed at the toplevel
  of the design will get benefit from dataflow optimization.

9. Scrolling down into the *Area Estimates*, observe that the number of BRAM_18K required at the top-level remained at 3.

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	-	-	-
Expression	-	-	0	38
FIFO	-	-	-	-
Instance	2	8	1036	1603
Memory	1	-	256	16
Multiplexer	-	-	-	72
Register	-	-	8	-
Total	3	8	1300	1729
Available	280	220	106400	53200
Utilization (%)	1	3	1	3

Resource estimate with DATAFLOW directive applied

10. Look at the console view and notice that **dct_coeff_table** is automatically partitioned in dimension *2*.
11. The *buf_2d_in* and *col_inbuf* arrays are partitioned as we had applied the directive in the previous run. The dataflow is applied at the top-level which created channels between top-level functions read_data, dct_2d, and write_data.

```
INFO: [XFORM 203-712] Applying dataflow to function 'dct' (dct.c:78), detected/extracted 3 process function(s):
        'read_data'
        'dct_2d'
        'write_data'.
INFO: [XFORM 203-11] Balancing expressions in function 'dct_1d' (dct.c:4)...8 expression(s) balanced.
INFO: [HLS 200-111] Finished Pre-synthesis Time (s): cpu = 00:00:01 ; elapsed = 00:00:06 . Memory (MB): peak =
INFO: [XFORM 203-541] Flattening a loop nest 'WR_Loop_Row' (dct.c:71:67) in function 'write_data'.
INFO: [XFORM 203-541] Flattening a loop nest 'RD_Loop_Row' (dct.c:59:67) in function 'read_data'.
INFO: [XFORM 203-541] Flattening a loop nest 'Xpose_Row_Outer_Loop' (dct.c:38:1) in function 'dct_2d'.
INFO: [XFORM 203-541] Flattening a loop nest 'Xpose_Col_Outer_Loop' (dct.c:49:1) in function 'dct_2d'.
INFO: [HLS 200-111] Finished Architecture Synthesis Time (s): cpu = 00:00:02 ; elapsed = 00:00:06 . Memory (MB)
INFO: [HLS 200-10] Starting hardware synthesis ...
INFO: [HLS 200-10] Synthesizing 'dct' ...
```

Console view of synthesis process after DATAFLOW directive applied

Perform performance analysis by switching to the Analysis perspective and looking at the dct performance profile view.

1. Switch to the *Analysis* perspective, expand the *Module Hierarchy* entries, and select the **dct_2d**
   entry.
2. Look at the *Performance Profile* pane.
   Observe that most of the latency and interval (throughput) is caused by the dct_2d function. The interval of the top-level function dct, is less than the sum of the intervals of the read_data, dct_2d, and write_data functions indicating that they operate in parallel and dct_2d is the limiting factor. From the Performance Profile tab it can be seen that dct_2d is not completely operating in parallel as Row_DCT_Loop and Col_DCT_Loop were not pipelined.

Module Hierarchy

	Negative Slack	BRAM	DSP	FF	LUT	Latency	Interval	Pipeline type
dct	0.65	3	8	1300	1729	508	375	dataflow
dct_2d	0.65	2	8	975	1242	374	374	none
write_data	-	0	0	32	188	66	66	none
read_data	-	0	0	29	173	66	66	none

Performance Profile | Resource Profile

	Pipelined	Latency	Iteration Latency	Initiation Interval	Trip count
dct_2d	-	374	-	374	-
Row_DCT_Loop	no	120	15	-	8
Xpose_Row_Outer_Loop_Xpose_Row_Inner_Loop	yes	64	2	1	64
Col_DCT_Loop	no	120	15	-	8
Xpose_Col_Outer_Loop_Xpose_Col_Inner_Loop	yes	64	2	1	64

Performance analysis after the DATAFLOW directive

One of the limitations of the dataflow optimization is that it only works on top-level loops and functions. One way to have the blocks in dct_2d operate in parallel would be to pipeline the entire function. This however would unroll all the loops and can sometimes lead to a large area increase.
An alternative is to raise these loops up to the top-level of hierarchy, where dataflow

optimization can be applied, by removing the dct_2d hierarchy, i.e. inline the dct_2d function.

3. Switch to the *Synthesis* perspective.

Apply INLINE Directive

Create a new solution by copying the previous solution (Solution5) settings. Apply INLINE directive to dct_2d. Generate the solution and analyze the output.

1. Select **Project > New Solution**.
2. A *Solution Configuration* dialog box will appear. Click the **Finish** button (with Solution5 selected).
3. Select the function *dct_2d* in the directives pane, right-click on it and select **Insert Directive...**
4. A pop-up menu shows up listing various directives. Select **INLINE** directive. The INLINE directive causes the function to which it is applied to be inlined: its hierarchy is dissolved.
5. Click on the *Synthesis* button.
6. When the synthesis is completed, the synthesis report will be opened.
7. Observe that the latency reduced from *508* to *495* clock cycles, and the Dataflow pipeline throughput drastically reduced from *375* to *114* clock cycles.
8. Examine the synthesis log to see what transformations were applied automatically.

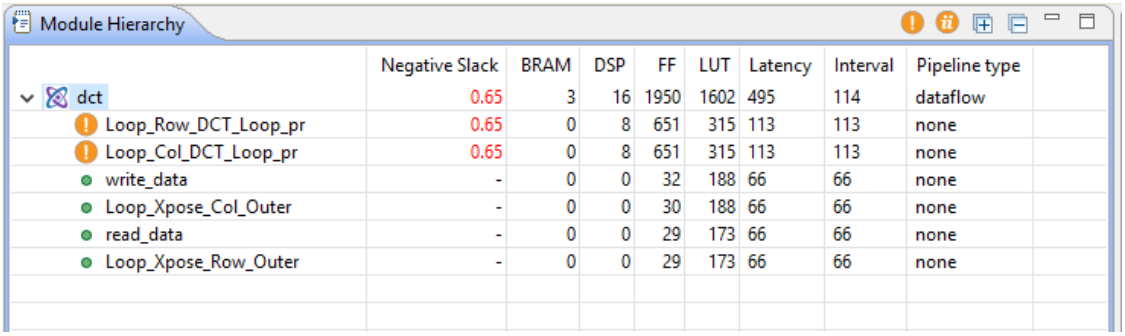
- The dct_1d function calls are now automatically inlined into the loops from which they are called, which allows the loop nesting to be flattened automatically.
- Note also that the DSP48E usage has doubled (from 8 to 16). This is because, previously a single instance of dct_1d was used to do both row and column processing; now that the row and column loops are executing concurrently, this can no longer be the case and two copies of dct_1d are required: Vivado HLS will seek to minimize the number of clocks, even if it means increasing the area.
- BRAM usage has increased once again (from 4 to 6), due to ping-pong buffering between more dataflow processes.

```
INFO: [XFORM 203-712] Applying dataflow to function 'dct' (dct.c:78), detected/extracted 6 process function(s):
        'read_data'
        'Loop_Row_DCT_Loop_proc'
        'Loop_Xpose_Row_Outer_Loop_proc'
        'Loop_Col_DCT_Loop_proc'
        'Loop_Xpose_Col_Outer_Loop_proc'
        'write_data'.
INFO: [XFORM 203-602] Inlining function 'dct_1d' into 'Loop_Row_DCT_Loop_proc' (dct.c:33->dct.c:87) automatically.
INFO: [XFORM 203-602] Inlining function 'dct_1d' into 'Loop_Col_DCT_Loop_proc' (dct.c:44->dct.c:87) automatically.
INFO: [XFORM 203-11] Balancing expressions in function 'Loop_Row_DCT_Loop_proc' (dct.c:13:61)...8 expression(s) balanced.
INFO: [XFORM 203-11] Balancing expressions in function 'Loop_Col_DCT_Loop_proc' (dct.c:13:61)...8 expression(s) balanced.
```

Console view after INLINE directive applied to dct_2d

9. Switch to the *Analysis* perspective, expand the *Module Hierarchy* entries, and select the **dct** entry.
   Observe that the dct_2d entry is now replaced with dct_Loop_Row_DCT_Loop_proc, dct_Loop_Xpose_Row_Outer_Loop_proc, dct_Loop_Col_DCT_Loop_proc, and dct_Loop_Xpose_Col_Outer_Loop_proc since the dct_2d function is inlined. Also observe that

all the functions are operating in parallel, yielding the top-level function interval (throughput) of 106 clock cycles.



	Negative Slack	BRAM	DSP	FF	LUT	Latency	Interval	Pipeline type
dct	0.65	3	16	1950	1602	495	114	dataflow
Loop_Row_DCT_Loop_pr	0.65	0	8	651	315	113	113	none
Loop_Col_DCT_Loop_pr	0.65	0	8	651	315	113	113	none
write_data	-	0	0	32	188	66	66	none
Loop_Xpose_Col_Outer	-	0	0	30	188	66	66	none
read_data	-	0	0	29	173	66	66	none
Loop_Xpose_Row_Outer	-	0	0	29	173	66	66	none

Performance analysis after the INLINE directive

10. Close Vivado HLS by selecting **File > Exit**.

Conclusion

In this lab, you learned various techniques to improve the performance and balance resource utilization.
PIPELINE directive when applied to outer loop will automatically cause the inner loop to unroll. When a
loop is unrolled, resources utilization increases as operations are done concurrently. Partitioning memory
may improve performance but will increase BRAM utilization. When INLINE directive is applied to a
function, the lower level hierarchy is automatically dissolved. When DATAFLOW directive is applied, the
default memory buffers (of ping-pong type) are automatically inserted between the top-level functions and
loops. The Analysis perspective and console logs can provide insight on what is going on.

Answers

Answers for question 1:
Estimated clock period: **6.508 ns**
Worst case latency: **3959 clock cycles**
Number of DSP48E used: **1**
Number of BRAMs used: **5**
Number of FFs used: **278**
Number of LUTs used: **982**